

The High Cost of Repeated AI Reasoning

How guarded agentic compaction turns recurring read-only agent work into verified deterministic programs

A LinkedIn-ready technical article based on the paper by Reza Rahimi

The High Cost of Repeated AI Reasoning

How guarded agentic compaction turns repeated read-only agent work into verified deterministic programs - while refusing unsafe cases.

Based on: Compiling Recurrent Agent Workflows into Guarded Programs

50-75%

fewer provider requests

39.5-81.4%

fewer tokens

51.7-73.0%

lower latency

Core idea: accelerate only what evidence supports - and retire everything else.

Core message: optimize only the parts of the workflow that evidence can license.

Why repeated agent loops get expensive

Tool-using agents are strong at solving new problems, but many production workflows become repetitive. The same read-only evidence gathering steps show up again and again: fetch a record, inspect a label, read comments, classify an item, and then ask the model for another decision. Every repeated model turn adds latency, token usage, and cost.

That sounds like caching, but it is not. A later tool argument may depend on a previous observation, a nominally read-only step may still have a hidden side effect, and a workflow that looks safe in one trace can fail under a different permission, freshness, or contract constraint. In other words, speed is only useful when the faster path is still the right path.

"Traces establish recurrence, not admissibility."

What Guarded Agentic Compaction does

The paper introduces guarded agentic compaction (GAC), a trace-to-program compiler that looks for recurrent read-only prefixes and converts only the supported part into deterministic code. Everything that the evidence does not determine stays with the model.

The key design choice is conservative by construction: the default outcome is refusal. GAC compiles a prefix only when typed provenance, effect declarations, manifest compatibility, bounded synthesis, verification, and finite-sample admission all line up. If any part is missing, the baseline agent remains unchanged.

At a glance

Problem	Repeated read-only agent prefixes still pay model cost at every boundary.
Method	Trace-to-program compilation with typed provenance, effect barriers, and exact admission.
Safety rule	If evidence is incomplete, retire and keep the baseline agent.
Reported effect	Fewer requests, tokens, latency, and cost on real GitHub workflows.

The pipeline at a glance

Guarded agentic compaction: from traces to runtime

A compact compiler pipeline with multiple independent refusal points.

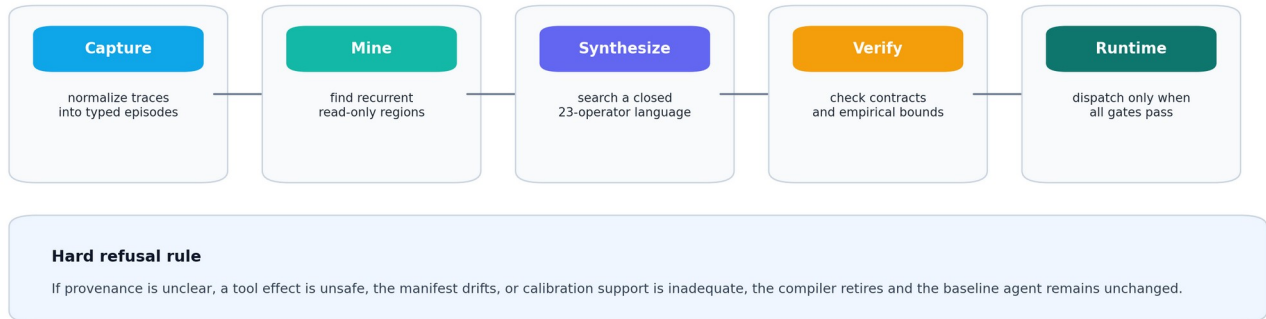


Figure 1. Guarded agentic compaction moves from trace capture to runtime gating, with multiple refusal points in between.

The system has four main stages. First, it captures traces into a typed episode representation. Second, it mines repeated candidate regions and synthesizes a small program from a closed 23-operator language. Third, it induces hard guards and verifiers, then calibrates a risk threshold using exact finite-sample bounds. Finally, runtime dispatch checks the guard, the score, and the manifest before execution.

That structure matters because it turns optimization into an audit trail. The compiler is not asked to prove that every future input will be correct. It is asked to show, with bounded evidence, that a specific prefix can be retired safely and that the baseline remains available whenever the support is not there.

What the experiments showed

On three real GitHub workflow families with live provider calls, compiled programs passed 90/90 exact contracts, while the unchanged agents passed 89/90. The paper reports no compiled-only failure and a pooled discordance upper bound of 3.3 percent. The efficiency gains were substantial: provider requests fell by 50.0-75.0 percent, tokens by 39.5-81.4 percent, latency by 51.7-73.0 percent, and estimated cost by 32.0-75.3 percent.

A wider cross-repository extension reached 580/580 exact discovery traces, completed 120/120 time-forward exact pairs on four repositories, and retired the fifth. A balanced rerun over three repositories reached 360/360 exact discovery traces and 180/180 exact held-out pairs, while reducing requests 66.7 percent, tokens 78.4 percent, observed wall latency 60.7 percent, and cost 72.7 percent.

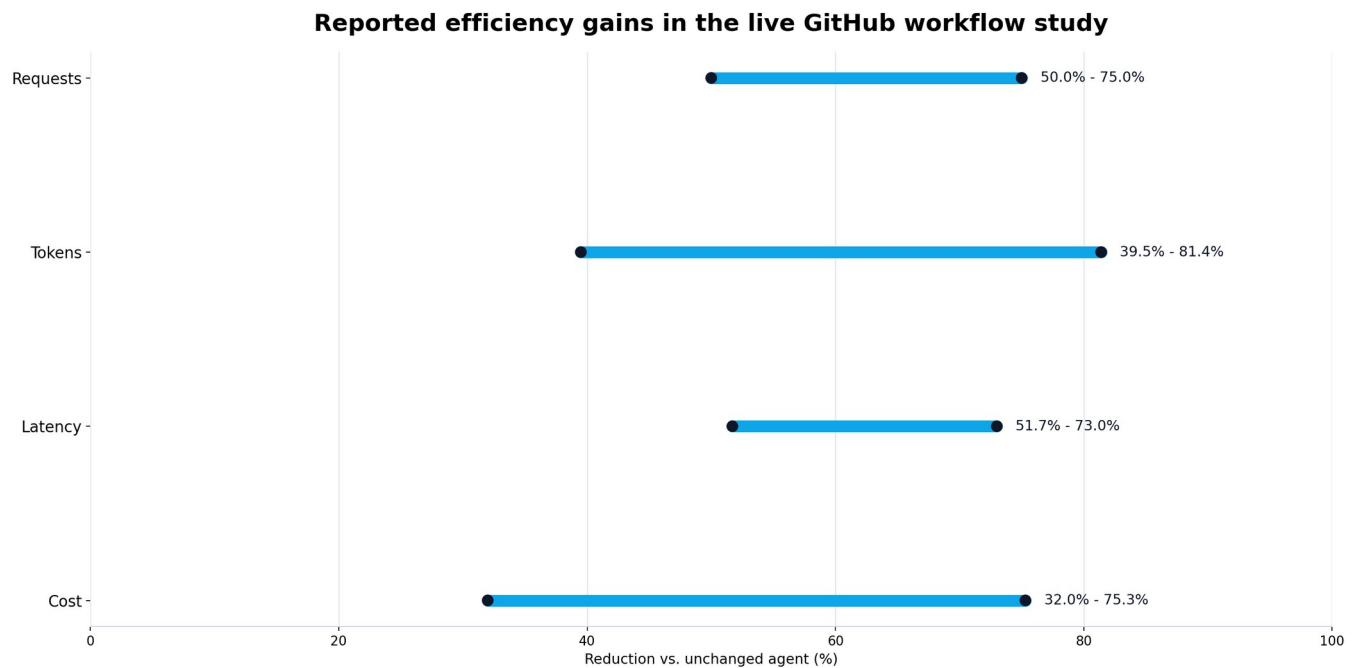


Figure 2. The paper reports substantial reductions in requests, tokens, latency, and cost across the live workflow study.

Selected headline results

Exact contracts	90/90 for compiled programs vs. 89/90 for unchanged agents
Compiled-only failures	None reported
Pooled discordance upper bound	3.3%
Cross-repository extension	580/580 exact discovery traces
Balanced rerun	360/360 exact discovery traces and 180/180 held-out pairs

What the results do not claim

The paper is careful not to overstate the result. It does not claim semantic equivalence, production certification, automatic API fusion, or universal superiority over hand-written code. It also shows that some benchmark families should retire because the evidence is not strong enough to justify compaction.

That is the part many optimizers miss. In this work, refusal is not a failure mode. Refusal is part of the product. The evidence boundary is the contribution: traces establish recurrence, not admissibility.

Boundaries worth keeping visible

Semantic equivalence	Not claimed
----------------------	-------------

Production certification	Not claimed
Automatic API fusion	Not claimed
Universal superiority over hand-written code	Not claimed

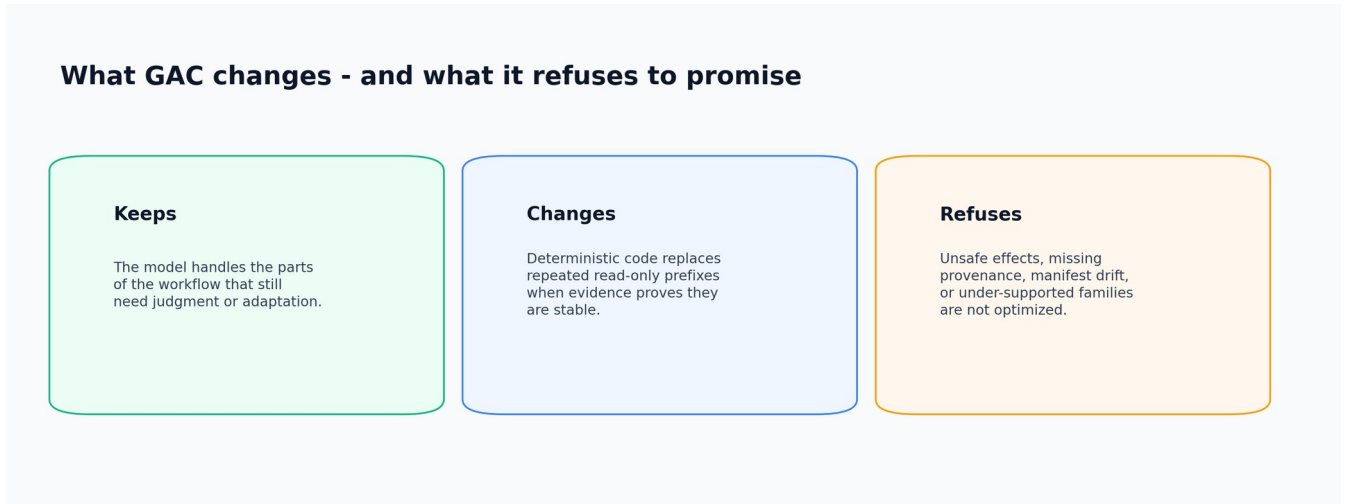


Figure 3. GAC keeps the model where judgment is needed, changes only the licensed deterministic prefix, and refuses unsafe cases.

Practical takeaway

For teams building AI agents, the lesson is simple but useful: do not optimize every boundary. Optimize only the boundaries that can be licensed by evidence, and preserve the baseline everywhere else. That keeps the system fast where it can be fast and honest where it must remain flexible.

If your workflow repeats the same read-only prefix, GAC offers a blueprint for turning that prefix into guarded code without removing the model from the loop where judgment is still needed.

Optimize only the boundaries that evidence can license. Keep the baseline everywhere else.

Based on the paper *Compiling Recurrent Agent Workflows into Guarded Programs* by Reza Rahimi.